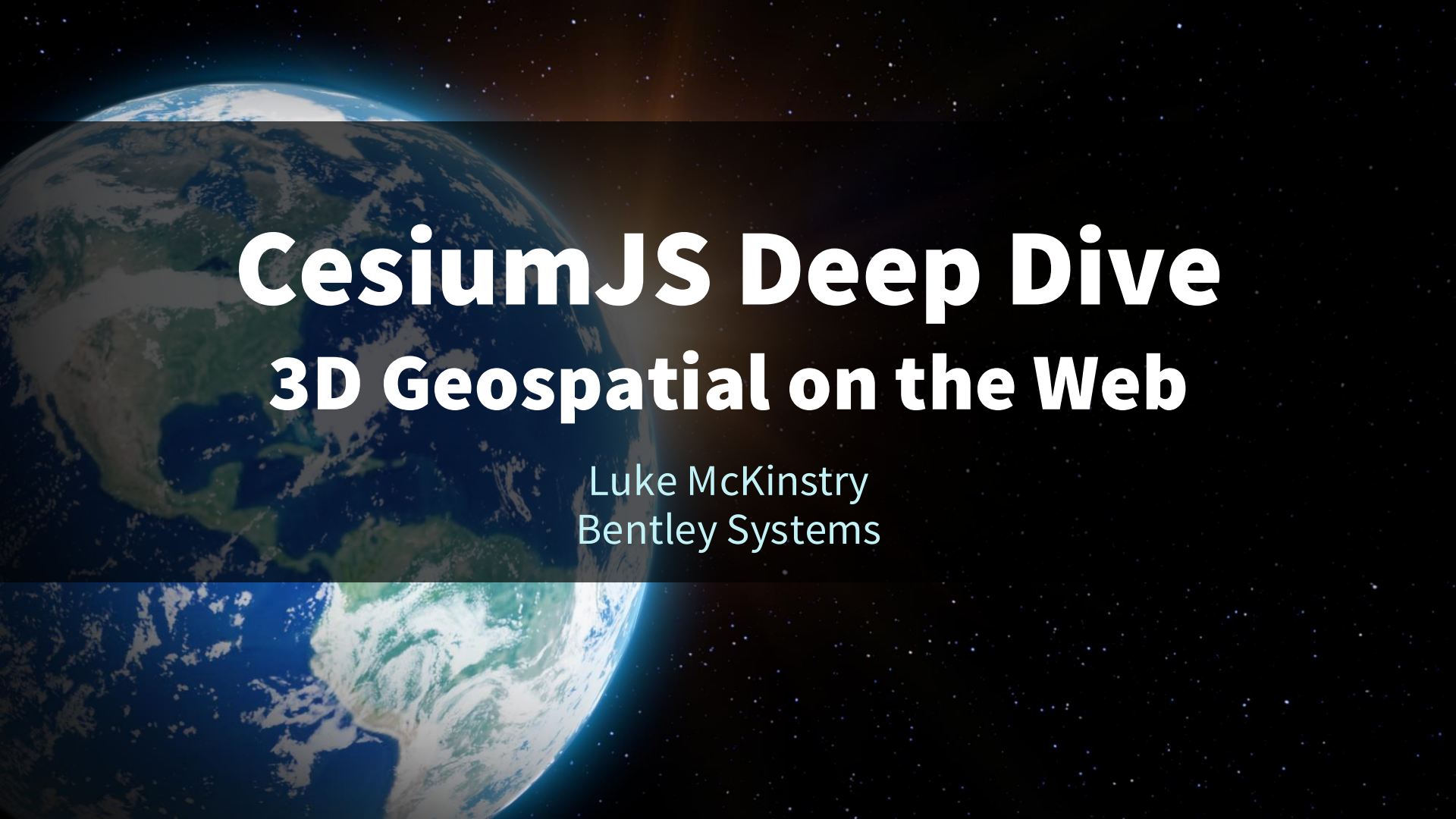




CesiumJS Deep Dive

3D Geospatial on the Web

Luke McKinstry
Bentley Systems

Agenda

10:40 – 11:00AM

Getting started

11:00 – 11:20AM

Your data in CesiumJS

11:20 – 11:40AM

Entities and animation

11:40AM – 12:00PM

Camera controls, finishing touches

GETTING STARTED



GETTING STARTED

Step 1: Setup your app

<https://github.com/CesiumGS/cesiumjs-workshop>

 cesiumjs-workshop Public

 Edit Pins

 Watch 0

 Fork 5

 Star 5

 main

 7 Branches

 1 Tag

 Go to file



 Add file



 Code

About

CesiumJS Deep Dive Workshop for the annual Cesium Developer Conference



 Readme

 Apache-2.0 license

 Code of conduct

 Contributing

 Activity

 Custom properties

 5 stars

 0 watching

 lukemckinsty	Merge pull request #9 from CesiumGS/ion-upload-fixup	19e426f · 1 hour ago	33 Commits
 .github/workflows	initial app setup	last year	
 .husky	initial app setup	last year	
 doc	add assets instructions in readme	yesterday	
 src	clarify gltf upload step	1 hour ago	
 .eslintignore	initial app setup	last year	
 .eslintrc.json	Documentation additions, package.json metadata, lock C...	last year	
 .gitignore	initial app setup	last year	



GETTING STARTED

Step 2: Connect your ion account

Navigate to ion.cesium.com and create a new access token

The screenshot shows the 'Access Tokens' page in the Cesium Ion web interface. The top navigation bar includes 'Stories', 'My Assets', 'Asset Depot', 'Access Tokens' (selected), and 'Usage'. On the right, there are links for 'What's new?', 'Support', and 'Learn'. The main content area is titled 'Access Tokens' with a help icon. Below the title is a 'Create token' button and a search bar. A table lists existing tokens:

Name	Last used	Scopes
Default Token	11/12/2021	assets:read, geocode
New Token	11/30/2021	assets:read, geocode

At the bottom of the table are 'Previous' and 'Next' navigation buttons. A sidebar on the right, titled 'New Token', contains an 'Edit' button, a 'Delete' button, a 'Name' input field (containing 'New Token'), a 'Token' input field (containing a long alphanumeric string), a 'View usage statistics' link, and an 'Access' section listing the scopes: 'assets:read' and 'geocode'.



GETTING STARTED

Step 2: Connect your ion account

Copy the new access token

```
// Step 1.2: Add your Cesium ion access token
Ion.defaultAccessToken = "your_ion_token_here";

// Step 1.3: Initialize the Cesium Viewer with the
// `cesiumContainer` ID
const viewer = new Viewer("cesiumContainer");
```

What's new? Support Learn

New Token

Edit Delete

Name

New Token

sInR5cCI6IkpXVCJ9.eyJqdGkiOiJhYT...
YtODExZi1jYzM3Njk2YjdlMWMiLCJpZC...
Z0DI5NzAxMX0.E2D4K_NnZUnrAvjhSjQ...
-FYVU

Access to the following scopes:



GETTING STARTED

Step 2: Connect your ion account



Cesium ion is free for community accounts, but you'll likely need a paid account when in production.



Attribution matters! Don't remove the ion logo if using ion data.



CESIUM ion Data attribution



GETTING STARTED

Step 2a: Add an Imagey Layer from the Asset Depot

- Open **Asset Depo**
- Find **Assure Maps Labels Only (Technology Preview)**
- Select **"Add"**
- View dataset listed in **My Assets**

The screenshot shows the Cesium Ion Asset Depot interface. The top navigation bar includes links for Stories, My Assets, Asset Depot (selected), Clips, Access Tokens, Usage, and Report. The main heading is 'Asset Depot' with a search bar. Below is a table of assets:

Name	Description	Type	Add
Aerometrex Denver High Resolution 3D Model with Street Level Enhanced 3D (Non-Commercial Trial)	Aerometrex generates its 3D models using advanced aerial acquisition and	3D Tiles	+
Aerometrex San Francisco High Resolution 3D Model with Street Level Enhanced 3D (Non-Commercial Trial)	Aerometrex generates its 3D models using advanced aerial acquisition and	3D Tiles	+
ArcticDEM Release 4	ArcticDEM is an NGA-NSF public-private initiative to automatically produce a	Terrain	-
Azure Maps Aerial (Technology Preview)	Map Tiles from Azure Maps. This asset is available as a Technology Preview and	Imagery	+
Azure Maps Labels Only (Technology Preview)	Map Tiles from Azure Maps. This asset is available as a Technology Preview and	Imagery	-



GETTING STARTED

Step 3: Build the app & start your local dev server

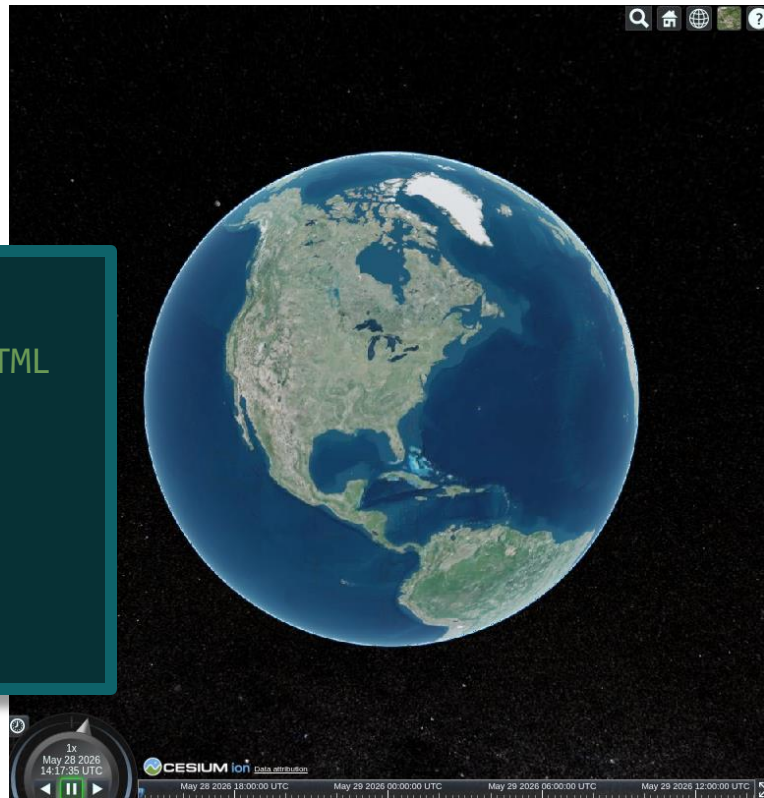
- `npm install`
- `npm run dev`
- In browser, go to <http://localhost:5173/>



GETTING STARTED

Step 4: Set up the Cesium Viewer

```
// Step 1.4: Initialize the Cesium Viewer in the HTML  
element with the `cesiumContainer` ID  
const viewer = new Viewer("cesiumContainer", {  
  infoBox: false,  
  geocoder: IonGeocoderProviderType.GOOGLE,  
});
```





GETTING STARTED

Step 5: Visualize Google Photorealistic 3D Tiles



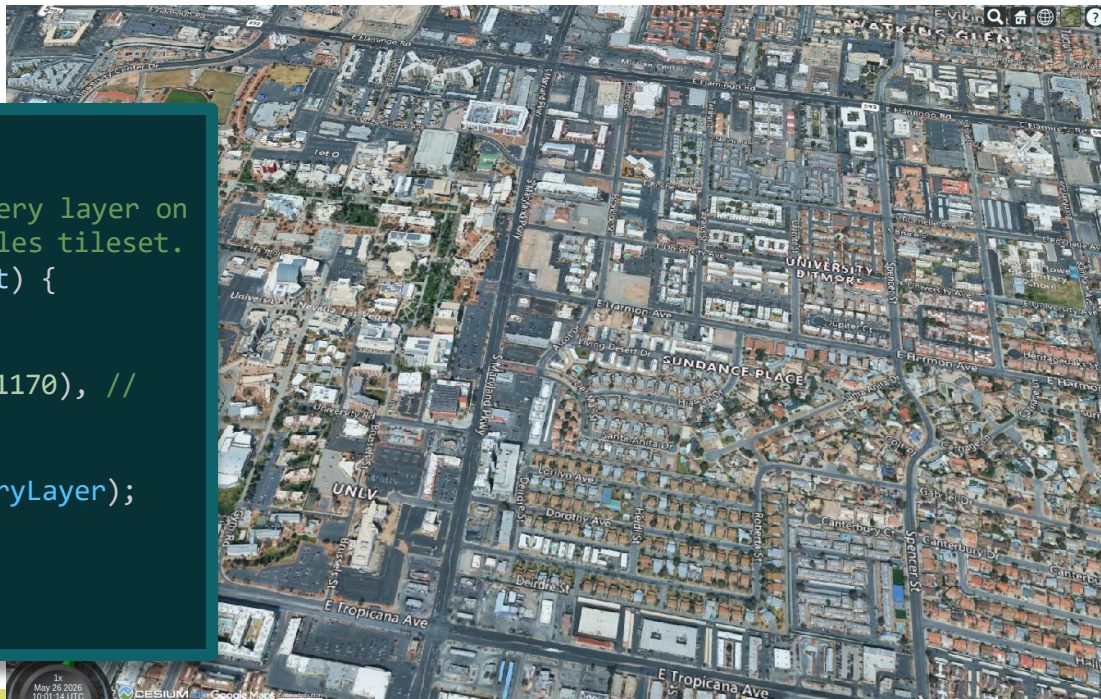
```
// Step 1.5: Add Google Photorealistic 3D Tiles, a global
photorealistic 3D tileset.
async function addGooglePhotorealistic3DTileset() {
  const tileset = await
createGooglePhotorealistic3DTileset({
  ...
});
viewer.scene.primitives.add(tileset);
return tileset;
}
const tileset = await addGooglePhotorealistic3DTileset();
```



GETTING STARTED

Step 6: Drape Imagery Overlay on 3D Tiles

```
// Step 1.6: Drape labels overlay imagery layer on  
// top of the Google Photorealistic 3D Tiles tileset.  
async function addLabelsOverlay(tileset) {  
  const labelImageryLayer = await  
    ImageryLayer.fromProviderAsync(  
      IonImageryProvider.fromAssetId(3891170), //  
      Ion asset ID for imagery  
    );  
  tileset.imageryLayers.add(labelImageryLayer);  
}  
await addLabelsOverlay(tileset);
```





GETTING STARTED

Step 7: Fly camera to location

```
// Step 1.7: Fly the camera to Las Vegas and orient the camera at the given heading, pitch & roll
function setCamera() {
  viewer.camera.flyTo({
    destination: Cartesian3.fromDegrees(-115.1494, 36.1071, 1172.6851),
    orientation: {
      heading: CesiumMath.toRadians(298.8682973473617),
      pitch: CesiumMath.toRadians(-12.38970963885237),
      roll: CesiumMath.toRadians(359.99975446786647),
    },
    duration: 3,
  });
}
setCamera();
```

YOUR DATA IN CESIUMJS



YOUR DATA IN CESIUMJS

Data types and formats

- [3D buildings](#)
- [AEC models](#)
- [BIM, CAD, or Other 3D model](#)
- [Photogrammetry or LiDAR-derived mesh](#)
- [Point clouds](#)
- [Satellite or drone imagery](#)
- [Terrain](#)

<https://cesium.com/learn/3d-tiling/tiler-data-formats/>

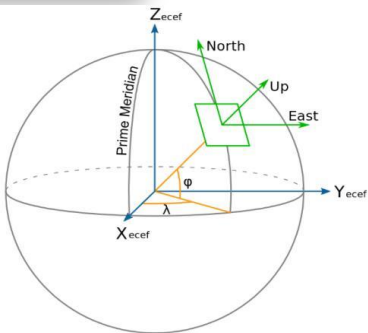


YOUR DATA IN CESIUMJS

Step 1: Upload a model



Cesium Earth-Centered Earth-Fixed (ECEF) Cartesian Coordinate System



```
const position = Cartesian3.fromDegrees(-115.161202, 36.109904, 500);
function addModel(position) {
  const heading = CesiumMath.toRadians(135);
  const pitch = 0;
  const roll = 0;
  const hpr = new HeadingPitchRoll(heading, pitch, roll);
  const orientation = Transforms.headingPitchRollQuaternion(position, hpr);
  viewer.entities.add({
    name: "CesiumBalloon",
    position: position,
    orientation: orientation,
    model: {
      uri: "./src/CesiumBalloon.glb",
    },
  });
  addModel(position);
}
```




YOUR DATA IN CESIUMJS

Step 1: Upload a model

A screenshot of the CesiumJS web application interface. The top navigation bar includes links for 'Stories', 'My Assets', 'Asset Depot', 'Access Tokens', and 'Usage'. The 'My Assets' section is active, displaying a header with a question mark and a blue 'Add data' button. Below the button, it shows '3.28 GB of 5.00 GB used' and a link to 'Get more storage'. A modal dialog is open, asking 'What kind of data is this?' with a text input field containing '3D Model (convert to glTF)'. At the bottom of the modal, there is a checkbox labeled 'Make available for download'. The background shows a table with columns for 'Name', 'Type', and 'Date added'.



YOUR DATA IN CESIUMJS

Step 1: Load Balloon from Cesium ion

Change the following 2 lines to load the balloon glTF from Cesium ion instead of a local file



```
const position = Cartesian3.fromDegrees(-115.161202, 36.109904, 500);
const resource = await IonResource.fromAssetId( your asset id here );
function addModel() {
    ...
    viewer.entities.add({
        name: "CesiumBalloon",
        ...
        model: {
            uri: resource
            ...
        },
    });
}
addModel();
```



YOUR DATA IN CESIUMJS

Step 2: Upload GeoJSON Polyline



```
// Step 2.2: Load GeoJSON of race course and add to scene as
polyline entity
async function addRaceCoursePath() {
  const response = await fetch("./src/lasvegas-street-
circuit.geojson");
  const geojson = await response.json();
  const coordinates =
geojson.features[0].geometry.coordinates;
  viewer.entities.add({
    polyline: {
      ...
    },
  });
  return coordinates;
}
const coordinates = await addRaceCoursePath();
```

ENTITIES AND ANIMATION



ENTITIES AND ANIMATION

Step 1: Sample Terrain Heights



```
// Step 3.1: Sample terrain heights along the race course to turn
coordinates into 3D positions that vehicles can follow
async function getTerrainSampledPositions(coordinates) {
  // Convert coordinates to cartographics
  const cartographics = coordinates.map((coord) =>
    Cartographic.fromDegrees(coord[0], coord[1]),
  );
  const terrainProvider = await createWorldTerrainAsync();
  // Sample terrain
  const sampled = await sampleTerrainMostDetailed(
    terrainProvider,
    cartographics,
  );
  return sampled;
}
```




ENTITIES AND ANIMATION

Step 2: Build Sampled Position Property



```
// Step 3.2: Build sample position property, an array of position samples
// with timestamps, that a vehicle can follow
function buildSampledPositionProperty(...) {
  const positionProperty = new SampledPositionProperty();
  let elapsedSeconds = 0;
  sampled.forEach((p) => {
    const position =
      Cartesian3.fromRadians(p.longitude, p.latitude, p.height);
    ...
    const time = JulianDate.addSeconds(
      start,
      elapsedSeconds + followDelaySeconds,
      new JulianDate()
    );
    positionProperty.addSample(time, position);
    previousPosition = position;
  });
}
```



ENTITIES AND ANIMATION

Step 3: Create a Vehicle

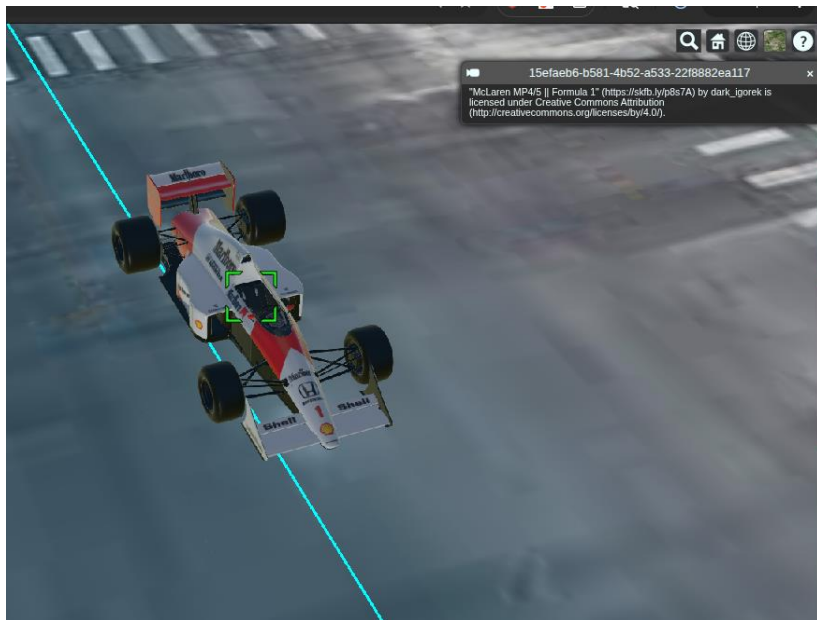


```
// Step 3.3: Create a vehicle following the race course,
// with a configurable follow delay to simulate a race
const vehicle = viewer.entities.add({
  availability: ...,
  position: positionProperty,
  orientation: new
  VelocityOrientationProperty(positionProperty),
  model: {
    uri: glbUri,
    minimumPixelSize: 32,
    maximumScale: 10,
  },
  label: ...,
  description: `
    "McLaren MP4/5 || Formula 1" (https://skfb.ly/p8s7A) by
    dark_igorek is licensed under Creative Commons Attribution
    (http://creativecommons.org/licenses/by/4.0/).`
});
```



ENTITIES AND ANIMATION

Step 4: Add Vehicles to Scene



```
// Step 3.4 Add multiple vehicles with different follow
// delays to the scene
async function addMovingVehicles(coordinates) {
  const sampled = await
  getTerrainSampledPositions(coordinates);
  // Lead vehicle
  await createMovingVehicle({
    ...
    glbUri: "./src/mclaren_mp45__formula_1.glb",
    labelText: "Car 1",
    followDelaySeconds: 0,
  });
  // Second vehicle 3 seconds behind
  await createMovingVehicle({labelText: "Car 2",
  followDelaySeconds: 3, ...});
  // Third vehicle 6 seconds behind
  await createMovingVehicle({labelText: "Car 3",
  followDelaySeconds: 6, ...});
  return;
}
await addMovingVehicles(coordinates);
```




ENTITIES AND ANIMATION

Step 5: Attribution & the Infobox



```
const viewer = new Viewer("cesiumContainer", {
  infoBox: true,
});

viewer.entities.add({
  ...
  model: ...
  description: `McLaren MP4/5 || Formula 1"
(https://skfb.ly/p8s7A) by dark_igorek is
licensed under Creative Commons Attribution
(http://creativecommons.org/licenses/by/4.0/).`
});
```



ENTITIES AND ANIMATION

Step 6: Add labels



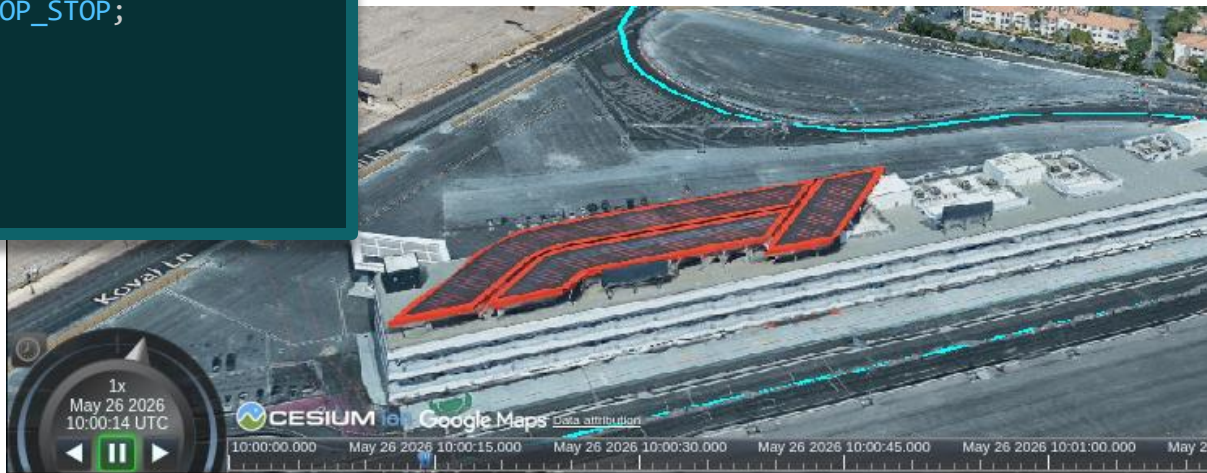
```
// Create entity with label
const vehicle = viewer.entities.add({
  availability: ...
  position: ...
  orientation: ...
  model: ...
  label: {
    text: labelText,
    font: "18px sans-serif",
    style: LabelStyle.FILL_AND_OUTLINE,
    outlineWidth: 2,
    verticalOrigin: VerticalOrigin.BOTTOM,
    pixelOffset: new Cartesian2(0, -50),
    showBackground: true,
    backgroundColor: Color.BLACK.withAlpha(0.7),
    fillColor: Color.WHITE,
  },
  description: ...
});
```



ENTITIES AND ANIMATION

Step 7: Set the Clock & Timeline

```
// Configure the clock and timeline
viewer.clock.startTime = start.clone();
viewer.clock.stopTime = stop.clone();
viewer.clock.currentTime = start.clone();
viewer.clock.clockRange = ClockRange.LOOP_STOP;
viewer.clock.shouldAnimate = true;
viewer.timeline.zoomTo(
  viewer.clock.startTime,
  viewer.clock.stopTime
);
```



CAMERA CONTROLS



CAMERA CONTROLS

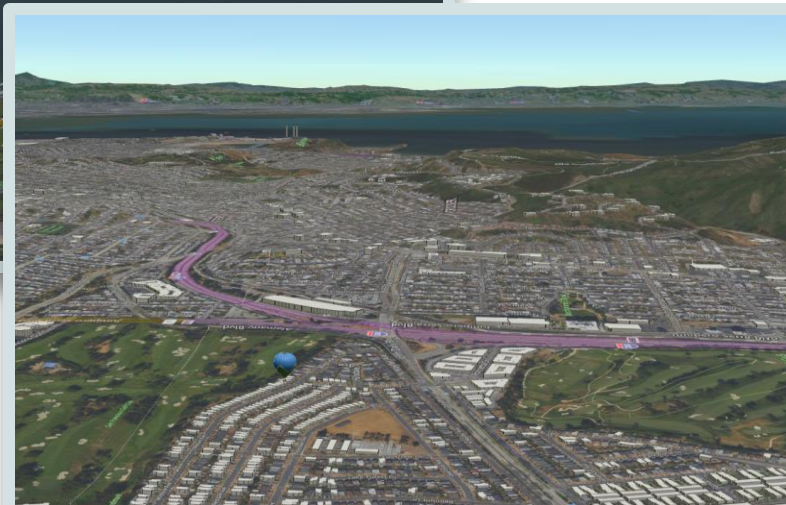
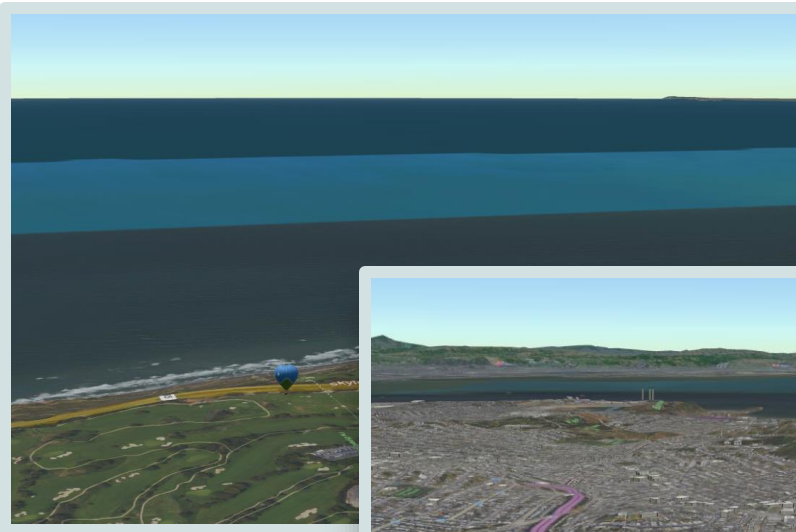
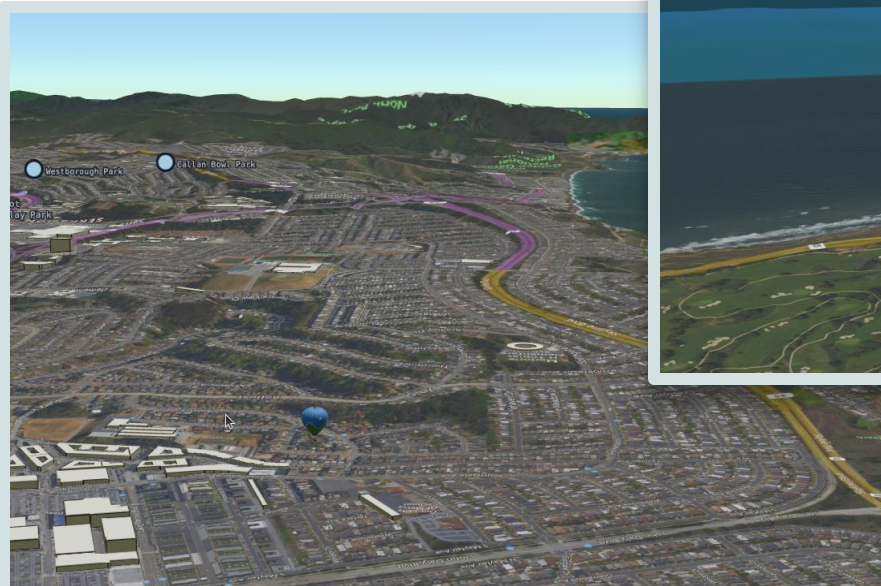
Step 0: Understand geocoding





CAMERA CONTROLS

Step 1: Orbit a point



Finishing Touches

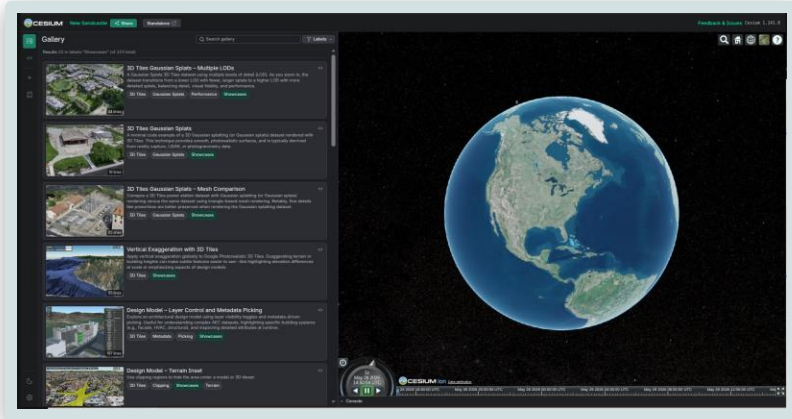


FINISHING TOUCHES

sandcastle.cesium.com

Ideas:

- Stream live vehicle positions
- Add custom 3D Data, eg Gaussian Splats
- Add picking behaviors
- Run video ads on the Las Vegas Sphere
 - <https://sandcastle.cesium.com/?id=video>



cesium.com/learn/cesiumjs/ref-doc/

NEXT STEPS

